INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA

DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE ENERO- JUNIO 2026

INGENIERÍA EN SISTEMAS COMPUTACIONALES

UNIDAD 1

ACOSTA MARTINEZ MOISES - 22210278

DOCENTE:

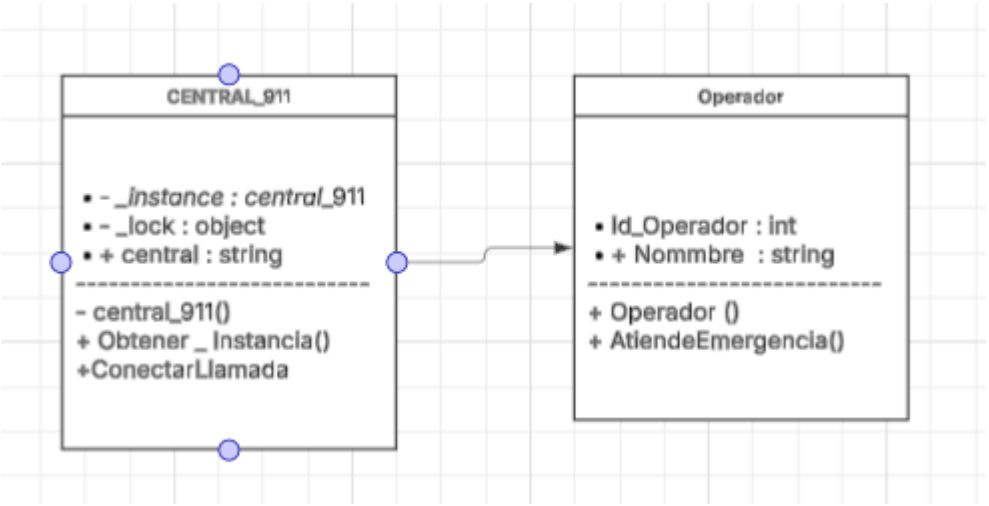
MARIBEL GUERRERO LUIS

09 DE Marzo DE 2026

Introducción

Tomando como base el patrón de diseño Singleton, se sabe que este ayuda a organizar el código y, como otros patrones, a resolver problemas comunes en programación. Este patrón se utiliza cuando se necesita que exista una sola instancia de una clase en todo el programa. En este caso, se aplica a una central 911, permitiendo que todas las llamadas y operadores trabajen con la misma central para que el sistema funcione correctamente.

DIAGRAMA UML



CÓDIGO

```
public class Central_911
{
   private static Central_911 _instance;
   private static readonly object _lock = new object();

   public string Central { get; private set; }

   private Central_911()
   {
      Central = "Central 911";
   }
```

```
    public static Central_911 Obtener_Instancia()
    {
        if (_instance == null)
        {
            lock (_lock)
            {
                if (_instance == null)
                {
                    _instance = new Central_911();
                }
            }
        }
        return _instance;
    }

    public void ConectarLlamada(Operador operador, string tipoEmergencia)
    {
        Console.WriteLine("\nLlamada conectada con el operador " + operador.Nombre);
        operador.AtiendeEmergencia(tipoEmergencia);
    }
}

public class Operador
{
    public int Id_Operador { get; set; }
    public string Nombre { get; set; }

    public Operador(int id, string nombre)
    {
        Id_Operador = id;
        Nombre = nombre;
    }

    public void AtiendeEmergencia(string tipoEmergencia)
    {
        Console.WriteLine($"Operador {Nombre} atendiendo emergencia de tipo:
{tipoEmergencia}");

        switch (tipoEmergencia)
        {
            case "Intento de suicidio":
                Console.WriteLine("Enviando unidades de apoyo y rescate");
                break;
            case "Incendio":
                Console.WriteLine("Enviando bomberos");
                break;
            case "Accidente":
```

```
            Console.WriteLine("Enviando paramedicos y oficiales");
            break;
        case "Violencia":
            Console.WriteLine("Enviando patrulla policial");
            break;
        case "Robo":
            Console.WriteLine("Enviando unidad policial inmediata");
            break;
        default:
            Console.WriteLine("Tipo de emergencia no reconocido.");
            break;
    }
  }
}

internal class Program
{
  static void Main(string[] args)
  {
    Central_911 central1 = Central_911.Obtener_Instancia();
    Central_911 central2 = Central_911.Obtener_Instancia();

    Operador op1 = new Operador(1, "Laura");
    Operador op2 = new Operador(2, "Carlos");
//AQUÍ HAY 2 OPERADORES MÁS
    Operador op3 = new Operador(3, "Ana");
    Operador op4 = new Operador(4, "Miguel");

//MAS LLAMADAS
    central1.ConectarLlamada(op1, "Incendio");
    central2.ConectarLlamada(op2, "Accidente");
    central1.ConectarLlamada(op3, "Robo");
    central2.ConectarLlamada(op4, "Violencia");
    central1.ConectarLlamada(op1, "Intento de suicidio");
    central2.ConectarLlamada(op2, "Incendio");

    Console.WriteLine("\nVerificando si es la misma instancia:");
    Console.WriteLine("ReferenceEquals: " + ReferenceEquals(central1, central2));

    Console.ReadKey();
  }
}
```

Identificar y explicar:

1.-el atributo estático de la instancia

- private static Central_911 _instance;
- se usa nada más para guardar la instancia de clase central_911
- static dice si pertenece a una clase
- obtener_instancia() verifica que _instance sea null
- si no hay se crea
- si ya hay la devuelve

2.-el constructor

```
private Central_911()
{
   Central = "Central 911";
}
```

inicializa objeto

el constructor es privado, por lo que no puede crear un objeto de otra parte

¿Por qué no se pueden crear múltiples objetos?

por que el constructor es privado, como se mencionó antes no se puede, pero específicamente no se puede usar “new ” de fuera de la clase

y como la clase usa un atributo estático devuelve la misma instancia, por eso solo existe un objeto siempre

VERIFICAR QUE ES UNA INSTANCIA

```
Console.WriteLine("ReferenceEquals: " + ReferenceEquals(central1, central2));
```

si el resultado es

ReferenceEquals: True

central 1 y 2 son exactamente el mismo objeto

código corriendo

```
C:\Users\moise\source\repos\

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Incendio
Enviando bomberos

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Accidente
Enviando paramedicos y oficiales

Llamada conectada con el operador Ana
Operador Ana atendiendo emergencia de tipo: Robo
Enviando unidad policial inmediata

Llamada conectada con el operador Miguel
Operador Miguel atendiendo emergencia de tipo: Violencia
Enviando patrulla policial

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Incendio
Enviando bomberos

Verificando si es la misma instancia:
ReferenceEquals: True
```

Conclusiones

llama la atención como una clase puede hacer que la creación de objetos sea reducida, viéndolo de cierta manera ayuda mucho a controla un solo acceso en este caso a un recurso dentro de la aplicación, y todo esto a lo largo de todo el programa.